basketbaLLM

An NBA history analyst agent

Zach Dvorak | AI Cohort project

Why build this?



The problem

LLMs state sports statistics with confidence, and often get them wrong. A model's memory is not a database.



Why it matters

Businesses want to ask questions of their own data in plain English. One wrong number costs trust.



Why you should care

This is a small, measurable version of that problem. The same method works on company data.

Schovia

What we built



Data

12 Kaggle CSVs covering 1947 to 2026, cleaned into DuckDB views



Agent

An OpenAI or Gemini model writes SQL, runs it, and explains the result



Evaluation

42 questions with gold answers, split into dev and test sets



App

A chat interface, designed and written by the models themselves

Schovia

How the system works



The model never recalls a statistic. It finds one by querying, so every number can be checked against the data.

Schovia

What is in the data?

80

seasons

1946-47 to 2025-26, including the BAA and ABA

5,416

players

every player in the dataset

27,011

player-seasons

after merging traded players

1,818

team-seasons

records, ratings and pace

What it cannot answer: the data is season-level. It has no individual games, playoff player stats, salaries or injuries. The agent is told to say so instead of guessing.

Schovia

Data traps that fool an AI

Trap	What goes wrong	Our fix
Season = the year it ends	2023-24 is stored as 2024, so asking for 2023 returns the wrong year	A season_label column, plus a rule in the prompt
Traded players appear twice	Leaderboards list a player twice and career totals double count	player_season keeps one row per player per season
Missing values are the text NA	Number columns load as text and averages break	The loader reads NA as a real null
'playoffs' means made the playoffs	It looks like playoff stats but is not	Renamed to made_playoffs
Rounded values tie	Jordan 30.12 and Chamberlain 30.07 both round to 30.1	Never round before sorting

Schovia

Techniques and tools

What we used

DuckDB
SQL straight on CSVs, no server

OpenAI and Gemini
function calling through their SDKs

Gradio
the chat interface

pytest
43 offline tests

What is interesting

SQL guard
read-only, one statement, 20 s timeout

Views as a semantic layer
friendly names and fixed traps

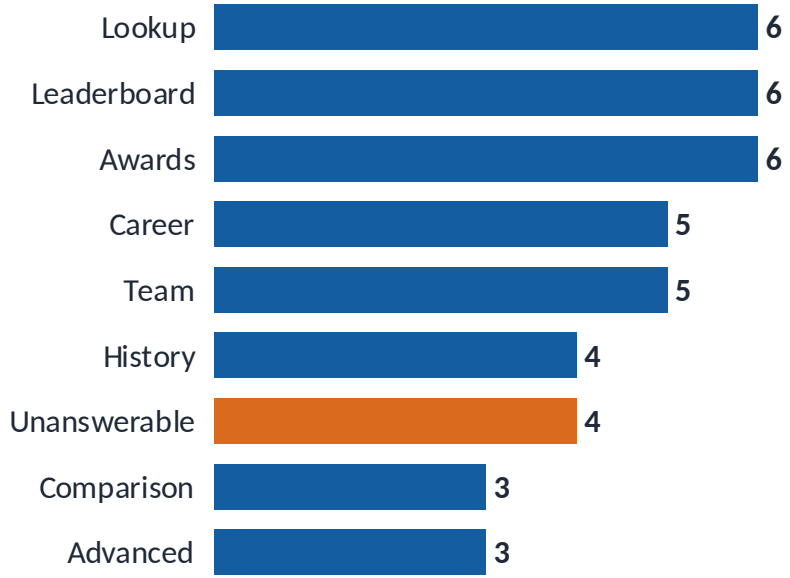
Retry with backoff
for free-tier rate limits

Fake model clients
tests run with no API keys

Schovia

Evaluation: how we measure

42 questions, by category



28 dev: tune against these

14 test: held back

Gold SQL runs on the same data the agent uses

Two graders: automatic match, and a judge from the other provider

Orange bar: questions the agent must decline

Schovia

What we have verified so far

Checked and working

- All 12 files load in Colab (Advanced.csv: 33,339 rows)
- 43 offline tests pass
- Agent loops tested with fake OpenAI and Gemini clients
- Gold answers checked against known facts

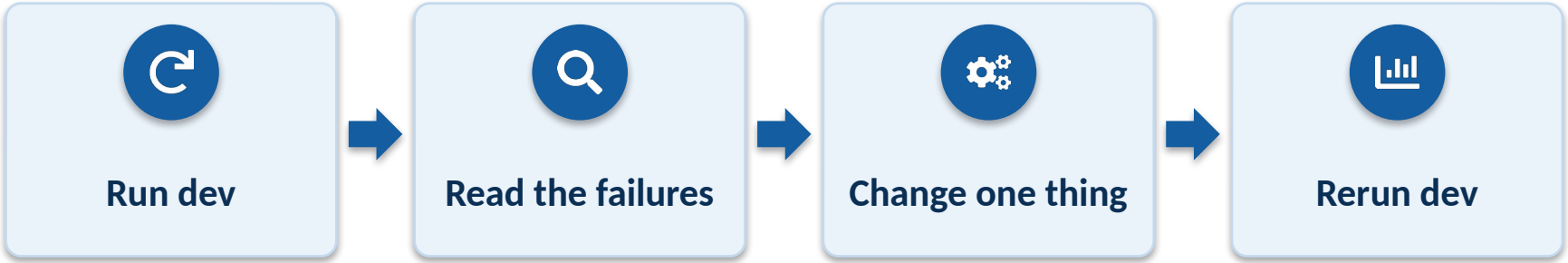
Gold answers from the data

Luka Dončić, 2023-24 scoring	**33.9 ppg**
Stephen Curry, 2015-16 threes	**402**
Best record, 2015-16	**Warriors, 73-9**
Most triple-doubles in a season	**Westbrook, 42**
Career ppg leader, 10,000+ points	**Jordan, 30.1**

Next: the first live accuracy run, OpenAI against Gemini on the 28 dev questions

Schovia

Evaluating does not train the model



Overfitting

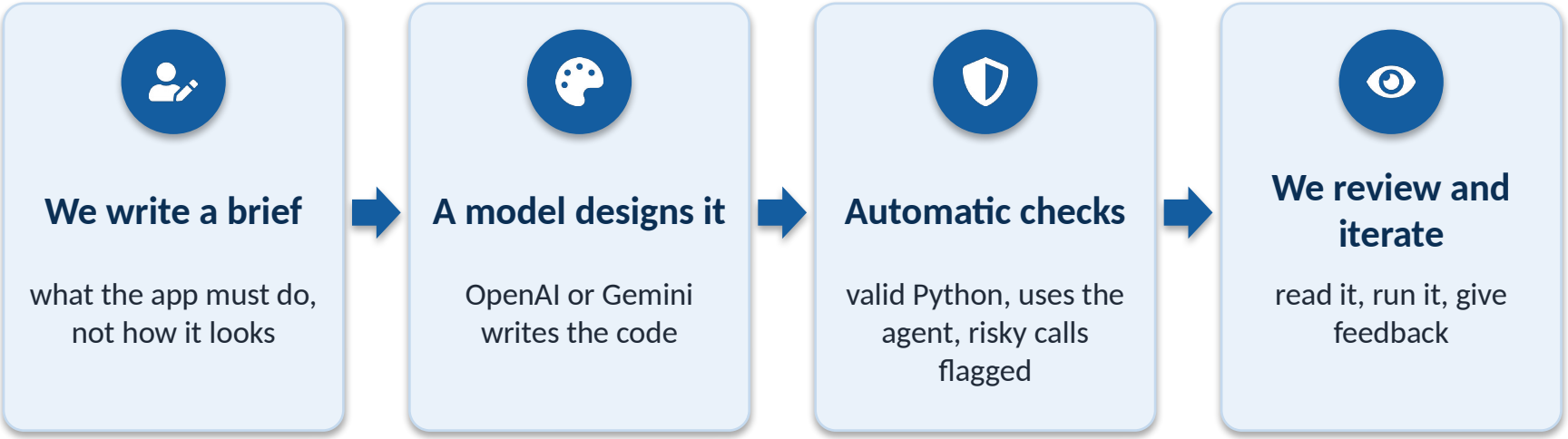
Tuning on the same questions fits those questions. Run the held-out test set once, at the end.

Noise

Answers vary between runs. Average two or three runs before believing a change.

Schovia

The models build the app



Layout, colors and typography are the model's call. Ask both providers and compare the two apps.

Schovia

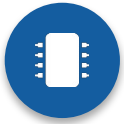
Demo: the scripted flow

1. **Who led the NBA in scoring in 2023-24?**
   Expect Luka Dončić, 33.9 per game, with the filters named
2. **Compare Jordan and LeBron by career per-game stats**
   Career totals without double counting trades
3. **How many 40-point games did Devin Booker have?**
   Expect a refusal: there are no individual games
4. **Open "How I got this"**
   Show the SQL the agent ran

Schovia


Demo

Future enhancements



Gemma as a third contender

An open model on a free Colab GPU, compared on the same eval



More of the dataset

Per 100 possessions, shooting and play-by-play files



Memory for follow-ups

So "what about Embiid?" works



Charts and fine-tuning

Draw the answer, or fine-tune a small model on question-to-SQL pairs

Schovia

Lessons learned



The data was harder than the model

End-year seasons, double-counted trades and NA text caused most risk



Evaluation measures, you improve

Change one thing, rerun dev, and hold out test



Structure beats a longer prompt

Views and a data dictionary give the model a clear map



Fake clients catch real bugs

Offline tests found a text-typed column and a path bug before Colab

Schovia

Questions



What would you ask the agent?

github.com/ZachDvorak/basketballm

Schovia


Thank You